



Universidad
Indoamérica

UNIDAD 3 · FUNDAMENTOS DE SISTEMAS OPERATIVOS

Archivos y sistema de archivos

Universidad Tecnológica Indoamérica

Laboratorio TD2

¿Cómo se accede a un archivo? Dos niveles

En el disco (persistente)

- Estructura del sistema de archivos: superbloque, tablas de inodos, bloques
- Inodos: los metadatos de cada archivo
- Directorios: listas *nombre* → *inodo*

En el núcleo (mientras está abierto)

- Tabla de descriptores, una por proceso
- Tabla de archivos abiertos, una para todo el sistema
- Tabla de inodos en memoria (y la caché de páginas)

Estructura del sistema de archivos



disco NVMe →

ext4: el grupo de bloques se repite a lo largo de la partición (con copias del superbloque)

Salida real

```
$ df -iT . | cut -c1-80; stat -f -c 'block size: %S bytes' .  
Filesystem      Type  Inodes   IUsed   IFree IUse% Mounted on  
/dev/nvme0n1p5 ext4 25600000 2372366 23227634   10% /  
block size: 4096 bytes
```

Un archivo = contenido + inodo

- El **inodo** guarda: tipo y permisos, enlaces duros, tamaño, dispositivo, UID/GID, marcas de tiempo y dónde están los bloques
- **No** guarda el nombre: el nombre está en el directorio
- **Corrección al original:** `ctime` no es la creación sino el último *cambio del inodo*. La creación (*Birth*) existe en ext4 y se lee con `statx()`.

Salida real

```
$ stat notas.txt
  File: notas.txt
  size: 11          Blocks: 8          IO Block: 4096
regular file
Device: 259,5   Inode: 12190686   Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/student)   Gid: (
1000/student)
Access: 2026-09-30 13:12:49.872600645 -0500
Modify: 2026-09-30 13:12:49.872600645 -0500
Change: 2026-09-30 13:12:49.879601063 -0500
 Birth: 2026-09-30 13:12:49.872600645 -0500
```

Del inodo a los bloques: punteros y extents

inodo → árbol de extents (profundidad 1, 340 entradas por bloque)

	lógico	longitud	físico
#0	0	32.768	34.816
#1	32.768	32.768	67.681
#2	65.536	32.768	100.546
#3	98.304	32.768	133.411
#4	131.072	32.768	166.276
#5	163.840	9.111	199.141

bloque físico =

$199.141 + (172.950 - 163.840)$

= 208.251

bloques de índice leídos: 1

el mismo archivo con punteros (ext3):

172.951 punteros en 170 bloques de índices · esta posición: 2 lecturas de índices · con extents: 6 × 12 bytes

676 MiB



el archivo en bloques lógicos (0 ... 172.951); cada franja es un extent

Un nombre apunta a un inodo

Salida real

```
$ ln notas.txt copia.txt; ln -s notas.txt atajo; ls -li notas.txt copia.txt atajo
12190687 lrwxrwxrwx 1 student student  9 Sep 30 13:12 atajo → notas.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 copia.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 notas.txt
```

Enlace duro (`link`)

Otra entrada al **mismo** inodo: mismo número, `nlink = 2`. No a directorios ni entre sistemas de archivos.

Enlace simbólico (`symlink`)

Un inodo **nuevo** cuyo contenido es una ruta (9 bytes: `notas.txt`). Puede quedar colgando.

Resolver rutas, enlazar y borrar

\$ cat atajo

ruta relativa: empieza en el directorio actual, inodo 1402

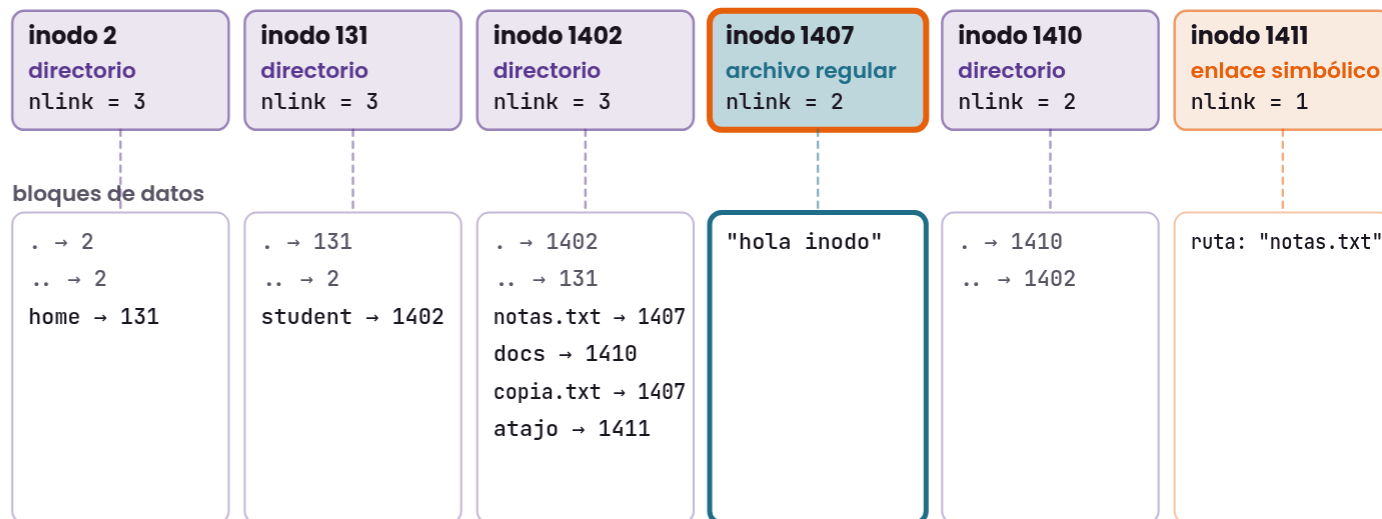
leer dir 1402: "atajo" → inodo 1411

enlace simbólico #1: continuar con "notas.txt"

leer dir 1402: "notas.txt" → inodo 1407

leer los bloques de datos del inodo 1407: "hola inodo"

tabla de inodos



"hola inodo"

Leer el inodo: stat, fstat, lstat

C

```
int stat(const char *path, struct stat *st);
int fstat(int fd, struct stat *st);
int lstat(const char *path, struct stat *st); /* does not follow links */

struct stat {
    dev_t     st_dev;      ino_t     st_ino;
    mode_t     st_mode;    nlink_t st_nlink;
    uid_t     st_uid;      gid_t     st_gid;
    dev_t     st_rdev;     /* device, if special file */
    off_t     st_size;     blkcnt_t st_blocks;
    blksize_t st_blksize; /* preferred I/O block size */
    struct timespec st_atim, st_mtim, st_ctim;
};
```

- 0 si va bien; -1 y `errno` si no:
- `EACCES` sin permiso de paso (`x`) en un directorio de la ruta
- `ENOENT` no existe · `ENOTDIR` un componente no es directorio
- `EL00P` más de 40 enlaces simbólicos
- 2026: tiempos con nanosegundos (`st_mtime = st_mtim.tv_sec`); `statx()` agrega la creación.

El tipo está en `st_mode` : macros `S_IS...`

<code>S_ISREG</code>	archivo regular	-
<code>S_ISDIR</code>	directorio	d
<code>S_ISLNK</code>	enlace simbólico	l
<code>S_ISCHR</code> ·	dispositivo de	c ·
<code>S_ISBLK</code>	caracteres · de bloques	b
<code>S_ISFIFO</code> ·	tubería · socket	p ·
<code>S_ISSOCK</code>		s

Salida real

```
$ ./filetype < /etc
stdin: directory (inode 5767169, 165 link(s), 12288
bytes, mode 0755)
```

Salida real

```
$ echo hi | ./filetype
stdin: FIFO (pipe) (inode 859154, 1 link(s), 0 bytes,
mode 0600)
```

Salida real

```
$ ./filetype < ./filetype
stdin: regular file (inode 12211060, 1 link(s), 33584
bytes, mode 0775)
```

Permisos: `access` ... y su trampa

C

```
int access(const char *path, int mode); /* R_OK W_OK X_OK F_OK */
```

- -1 si no existe o no hay permiso: `ER0FS` (solo lectura), `ETXTBSY` (ejecutable en uso)...
- Comprueba con el UID *real*, no el efectivo

Carrera TOCTOU (actualización 2026)

Entre `access()` y `open()` otro proceso puede cambiar el archivo por un enlace a `/etc/passwd`. La forma segura es **abrir y tratar el error** (`EACCES`), o `openat2()` con `RESOLVE_NO_SYMLINKS`. El TD8 (`agentd`) usa `realpath` y Landlock por esta razón.

Crear, enlazar y borrar nombres

C

```
int mknod(const char *path, mode_t mode, dev_t dev);
int mkfifo(const char *path, mode_t mode); /* S_IFIFO */
int link(const char *old, const char *new);
int symlink(const char *target, const char *path);
int unlink(const char *path); /* EISDIR */
int rmdir(const char *path); /* must be emp
```

- `mknod` crea cualquier tipo: el tipo va unido con OR a los permisos
(`S_IFREG|S_IRUSR|S_IXUSR` = `S_IFREG|0500`);
hoy se usa casi solo para dispositivos. Para archivos: `open(O_CREAT)` ; para tuberías: `mkfifo`.
- Los permisos pedidos pasan por la `umask` del proceso (típicamente `022` o `002`).
- `unlink` borra una *entrada*, no el archivo.

Abrir o crear: `open`

C

```
int open(const char *path, int flags, ... /* mode_t mode */);  
int openat(int dirfd, const char *path, int flags, ...);
```

- Acceso: `O_RDONLY`, `O_WRONLY`, `O_RDWR`
- Creación: `O_CREAT`, `O_EXCL` (falla si existe), `O_TRUNC`
- Estado: `O_APPEND` (cada `write` al final, de forma atómica)
- Devuelve el **menor descriptor libre** (≥ 0) o `-1`

Hoy además

- `O_CLOEXEC` : no heredar el descriptor en `exec` (glibc lo usa siempre, míralo en `strace`)
- `O_NOFOLLOW`, `O_DIRECTORY`, `O_TMPFILE`
- glibc implementa `open` con `openat(AT_FDCWD, ...)`; `creat()` es `open(..., O_WRONLY|O_CREAT|O_TRUNC)`

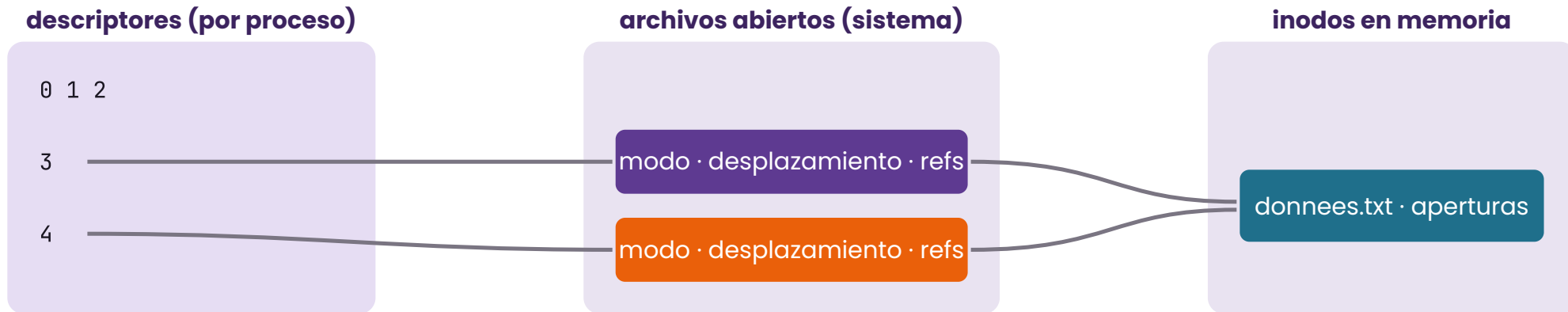
`close` : ¿cuándo desaparece un archivo?

Cuando `nlink = 0` y ningún descriptor lo tiene abierto. Hasta entonces, borrado pero vivo:

Salida real

```
$ exec 3< notas.txt; rm notas.txt copia.txt; readlink /proc/$$/fd/3; cat <&3; ls -l atajo; cat atajo
~/fso/tds/tests/tmp/td2/notas.txt (deleted)
hola inodo
lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
cat: atajo: No such file or directory
[exit status 1]
```

La gestión de archivos en el núcleo: tres tablas



El **desplazamiento** vive en la tabla del medio: por eso importa quién comparte cada entrada.

Alias: dos `open`, `dup` y `fork`

```
fd = open("donnees.txt", O_RDONLY);
read(fd, buf, 4); write(1, buf, 4);
pid = fork();
/* child */ read(fd, buf, 4); write(1, buf, 4);
/* parent */ wait(NULL); n = read(fd, buf, 4);
read(3) usa la entrada A: lee "efgh" y su desplazamiento pasa a 8
```

salida estándar:
"abcdefgh"

descriptores (por proceso)

padre

0	→ terminal
1	→ terminal
2	→ terminal
3	
4	

hijo

0	→ terminal
1	→ terminal
2	→ terminal
3	
4	

archivos abiertos (sistema)

entrada A

O_RDONLY
offset = 8
refs = 2

inodos en memoria (sistema)

donnees.txt

abierto por 1 entrada(s)

a b c d e f g h

Funciones básicas, `read` y `write` : repaso

- Inodos: `stat`, `chmod`, `chown`, `link`, `unlink` ...
- Descriptores: `open`, `read`, `write`, `lseek`, `close`, `fcntl` ...
- `fopen` / `fread` : biblioteca, no llamadas al sistema (`fileno()`)

`read` puede devolver menos, 0 al final, `-1/ EINTR`; `write` puede ser parcial. El ejemplo de la diapositiva 51 tenía un error: A2.4.

Recorrer directorios: `opendir`, `readdir`, `closedir`

C

```
DIR *opendir(const char *name);
struct dirent *readdir(DIR *dir); /* NULL at the end */
int closedir(DIR *dir);

struct dirent {
    ino_t      d_ino;      /* inode number */
    unsigned char d_type;  /* DT_REG, DT_DIR, DT_LNK... */
    char       d_name[256];
    /* d_off, d_reclen: internal */
};
```

- `readdir` devuelve las entradas en el orden interno del sistema de archivos (en ext4, el de un índice por hash), no alfabético
- Incluye `.` y `..`
- **Actualización:** el original decía "no uses los primeros cuatro campos". Hoy `d_ino` y `d_type` son útiles (ahorran un `stat`), pero `d_type` puede ser `DT_UNKNOWN`: entonces hay que llamar a `lstat`.

Recorrer un directorio: `myls -lR`

DIR *d = opendir(dir); y luego:

C tds/td2/myls.c

```
struct dirent *e;
while ((e = readdir(d)) != NULL) {
    if (n == cap) {
        cap *= 2;
        names = realloc(names, cap * sizeof *names);
        exit_if(names == NULL, "realloc");
    }
    names[n] = strdup(e->d_name);
    exit_if(names[n] == NULL, "strdup");
    n++;
}
exit_if(closedir(d) == -1, "closedir");
```

Salida real

```
$ ./myls -lR demo
demo:
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
.
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
..
-rw-rw-r--   1 student student     3 2026-09-30 13:12
a.txt
lrwxrwxrwx   1 student student     5 2026-09-30 13:12
link -> a.txt
prw-rw-r--   1 student student     0 2026-09-30 13:12
pipe
drwxrwxr-x   2 student student   4096 2026-09-30 13:12
sub

demo/sub:
drwxrwxr-x   2 student student   4096 2026-09-30 13:12
.
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
..
```

Un archivo especial: la FIFO del chat

servidor

read() = 0 → termina

cliente 1

cerró

cliente 2

cerró

- el servidor sale de open()
- C1 abre la FIFO para escribir
- C2 abre la FIFO para escribir
- C1 cierra
- C2 cierra
- read() = 0: cerró el último escritor → fin de archivo

chat.fifo · búfer del núcleo: 16 páginas × 4 KiB = 64 KiB



↓ read(fd, buf, 4096) → salida del servidor

hola a todos
 buenas
 ¿alguien en el lab?
 aquí el cliente 2
 chao
 me voy

read() = 0 → EOF

Lo que el original no alcanzó a contar

Diario

ext4 y XFS escriben primero en un diario: tras un corte de luz no hace falta revisar todo el disco.

Copia en escritura

Btrfs y ZFS nunca sobrescriben: instantáneas baratas y sumas de verificación de cada bloque.

Caché y `fsync`

`write` termina en la caché de páginas; solo `fsync` garantiza que llegó al disco.

"Todo es un archivo": `/proc`, `/sys`, `/dev` y FUSE usan la misma interfaz `open` / `read` / `write`.

TD2 resuelto + desafío con IA

- `filetype` con la terminal, un directorio, un archivo y una tubería
- El chat por FIFO, con dos terminales y luego con tres
- `alias_open` frente a `alias_dup`; `myls -lR`

Bash

```
make -C tds && tds/tests/run.sh td2
```



Universidad
Indoamérica

PARA LLEVAR

Tres preguntas

1. ¿Dónde está el nombre de un archivo y dónde su tamaño?
2. ¿Por qué `dup` imprime "abcdefgh" y dos `open` "abcdabcd"?
3. ¿Cuándo libera el núcleo los bloques de un archivo borrado?